

CO4 – AT2

Industry Problem Solving Task

SOURCE CODE:

Q2. TSP – Logistics Optimization

```
#include <stdio.h>

#define N 15

#define INF 999999

int main()
{
    int cost[N][N] = {
        {0, 29, 20, 21, 16, 31, 100, 12, 4, 31, 18, 23, 27, 14, 25},
        {29, 0, 15, 29, 28, 40, 72, 21, 29, 41, 32, 18, 35, 22, 30},
        {20, 15, 0, 15, 14, 25, 81, 9, 23, 27, 16, 20, 30, 17, 26},
        {21, 29, 15, 0, 4, 12, 92, 12, 25, 13, 18, 22, 28, 20, 15},
        {16, 28, 14, 4, 0, 15, 95, 10, 20, 16, 17, 19, 25, 18, 21},
        {31, 40, 25, 12, 15, 0, 90, 18, 30, 10, 25, 28, 35, 23, 27},
        {100, 72, 81, 92, 95, 90, 0, 88, 79, 85, 70, 75, 65, 80, 73},
        {12, 21, 9, 12, 10, 18, 88, 0, 15, 20, 13, 17, 24, 11, 19},
        {4, 29, 23, 25, 20, 30, 79, 15, 0, 28, 14, 16, 22, 9, 18},
        {31, 41, 27, 13, 16, 10, 85, 20, 28, 0, 22, 25, 31, 19, 24},
        {18, 32, 16, 18, 17, 25, 70, 13, 14, 22, 0, 15, 21, 12, 20},
```

```

    {23, 18, 20, 22, 19, 28, 75, 17, 16, 25, 15, 0, 26, 13, 21},
    {27, 35, 30, 28, 25, 35, 65, 24, 22, 31, 21, 26, 0, 18, 29},
    {14, 22, 17, 20, 18, 23, 80, 11, 9, 19, 12, 13, 18, 0, 16},
    {25, 30, 26, 15, 21, 27, 73, 19, 18, 24, 20, 21, 29, 16, 0}
};

int totalStates = 1 << (N - 1);

int **dp = malloc(totalStates * sizeof(int *));

int **parent = malloc(totalStates * sizeof(int *));

for (int i = 0; i < totalStates; i++)
{
    dp[i] = malloc(N * sizeof(int));

    parent[i] = malloc(N * sizeof(int));

    for (int j = 0; j < N; j++)
    {
        dp[i][j] = INF;

        parent[i][j] = -1;
    }
}

/* Base cases */

for (int j = 1; j < N; j++)
{
    int mask = 1 << (j - 1);

    dp[mask][j] = cost[0][j];

```

```

}

/* Dynamic Programming */

for (int mask = 1; mask < totalStates; mask++)
{
    for (int j = 1; j < N; j++)
    {
        if (!(mask & (1 << (j - 1))))
            continue;

        int previousMask = mask ^ (1 << (j - 1));

        if (previousMask == 0)
            continue;

        for (int k = 1; k < N; k++)
        {
            if (previousMask & (1 << (k - 1)))
            {
                int newCost = dp[previousMask][k] + cost[k][j];

                if (newCost < dp[mask][j])
                {
                    dp[mask][j] = newCost;

                    parent[mask][j] = k;
                }
            }
        }
    }
}

```

```

    }

}

/* Find final city */

int fullMask = totalStates - 1;

int minCost = INF;

int lastCity = -1;

for (int j = 1; j < N; j++)

{

    int tourCost = dp[fullMask][j] + cost[j][0];

    if (tourCost < minCost)

    {

        minCost = tourCost;

        lastCity = j;

    }

}

/* Reconstruct route */

int route[N + 1];

int mask = fullMask;

int current = lastCity;

for (int i = N - 1; i >= 1; i--)

{

    route[i] = current;

    int previous = parent[mask][current];

```

```
    mask = mask ^ (1 << (current - 1));

    current = previous;

}

route[0] = 0;

route[N] = 0;

printf("Optimal Route:\n");

for (int i = 0; i <= N; i++)

{

    printf("%d", route[i]);

    if (i < N)

        printf(" -> ");

}

printf("\n\nMinimum Travel Cost = %d\n", minCost);

for (int i = 0; i < totalStates; i++)

{

    free(dp[i]);

    free(parent[i]);

}

free(dp);

free(parent);

return 0;

}
```